MODULE DATASHEET / 模块数据手册

B1 图层叠加与字幕

OSD Caption Subsystem

osd_caption.v ▪ osd_strings.v ▪ osd_font8x16.v

器件	`Anlogic EG4S20BG256`（安路 EG4 系列 FPGA）
工程	SmartCampusMedia ——智播校园 HDMI 图片播放系统
分辨率	640 × 480 有效区，800 × 525 全帧 @ 59.5 Hz
像素时钟	25 MHz（`clk_pixel`）
字幕粒度	8 × 16 点阵，41 字形，16 行 × 64 列字符表
文档版本	v1.0
对应提交	B1 扩展项交付（仿真 + 综合布线已过，未上板）

本手册的定位。它不是「给评委看的成果说明」，而是**给要改这份代码的人看的说明书。**数据手册的写法是：先给引脚与参数，再给时序与真值表，最后才是功能描述——读者应该能只看第 3 节和第 4 节就把模块接进自己的设计，而不必先读懂全部 RTL。

目录

1 这份手册要解决什么问题 **1**

1.1 一句话定位 . . . 1

1.2 为什么值得单独写一份数据手册 . . . 1

2 系统位置与数据通路 **1**

2.1 在整个工程里的位置 . . . 1

2.2 三层职责 . . . 1

3 端口定义 **2**

3.1 `osd_caption`（字幕渲染器） . . . 2

3.2 `osd_strings` 与 `osd_font8x16` . . . 2

3.3 在 `video_overlay` 里怎么接 . . . 2

3.4 状态机编码到状态词的映射 . . . 2

4 屏幕坐标与版式 **4**

4.1 四块文字的位置 . . . 4

4.2 三个坐标设计决策 . . . 4

4.3 列号换算：最容易出错的一处 . . . 5

5 字形 ROM **5**

5.1 字符集：只有 41 个 . . . 5

5.2 字符集总表 . . . 5

5.3 为什么索引是 `char_code[5:0]` 而不是 `char_code - 0x20` . . . 7

5.4 为什么是 8×16 而不是 8×8 . . . 7

5.5 为什么不用 BRAM . . . 7

6 字符串表 **7**

6.1 16 行 × 64 列 . . . 7

6.2 各行内容 . . . 8

6.3 占位符 `#` 与 `*` . . . 8

7 滚动标语 **9**

7.1 按像素滚，不按字符格滚 . . . 9

7.2 三个必须踩对的点 . . . 9

7.3 滚动速度 . . . 9

8 单路渲染架构 **9**

8.1 核心决策 . . . 10

8.2 如果给每块文字各配一套 ROM 会怎样 . . . 10

9 衬底：为什么压暗而不是填黑 **10**

9.1 问题 . . . 10

9.2 衬底只覆盖文字实际占据的 x 区间 . . . 10
9.3 优先级链 . . . 11

10 为什么字幕层常接 `1'b1` 11

11 参数速查 11

12 验证 12
12.1 三档验证口径 . . . 12
12.2 逐点比对方法 . . . 12
12.3 预览图交叉校验 . . . 12
12.4 综合布线结果 . . . 13

13 生成脚本 13
13.1 为什么字模和字符串表都必须脚本生成 . . . 13
13.2 `gen_osd_font8.py` . . . 14
13.3 `gen_osd_strings.py` . . . 14
13.4 `gen_datasheet_assets.py`（本手册的图源） . . . 14

14 踩坑清单 14

15 还没做的 15

A 文件清单 15

B 字形来源与许可 15

1 这份手册要解决什么问题

1.1 一句话定位

这个子系统把「一块 640×480 的图像」变成「一块带文字的 640×480 图像」，文字与图像在同一拍像素时钟内合成，不需要帧缓冲、不需要行缓冲、不引入一拍延迟。

它解决的具体问题是：屏幕上要显示「SMART CAMPUS」标题、「IMG 07」这样的页码、「PLAY / PAUSE / EMERGENCY」这样的播放状态，以及底部一条缓慢左移的滚动标语。在它出现之前，这些信息只能用 12 个方块位图表示开关量，拼不出任何单词。

1.2 为什么值得单独写一份数据手册

字幕看起来是「画几个字」，实际动手会连续撞上五类问题，每一类都有反直觉的坑：

1. **字形数据从哪来。**手写 41 个 8×16 点阵是不可能的；从字体文件栅格化又有「8×8 糊成一团」的粒度陷阱（5 节）。
2. **坐标怎么换算成查表地址。**位切片 `active_x[8:3]` 看起来等价于 `(x-576)/8`，但在 $x = 576$ 时折返成 8，减法溢出——这个 bug **综合不报错、仿真不报错，只有上板才看出来**（4 节）。
3. **滚动为什么不能一格一格移。**按字符格滚动会看到「跳字」，必须按像素滚动，而按像素滚动时列内偏移必须用滚动后的坐标（7 节）。
4. **4 处文字要不要 4 套 ROM。**4 套 ROM 直接把 LUT 吃掉一大块，而本设计 LUT 余量只剩三成（8 节）。
5. **白字在白云上就消失了。**需要衬底，但填黑会糊出硬边黑条，压暗才保留画面层次（9 节）。

本手册把每一处都展开到「为什么这样写、换个写法会怎样」的粒度。

读法建议

要接线：读第 3 节（端口）与第 4 节（坐标）。**要改文案**：读第 6 节，然后改脚本重跑，不要手改 `.v`。**要改字形**：读第 5 节与第 13 节。**要理解设计取舍**：读第 8 节与第 14 节。

2 系统位置与数据通路

2.1 在整个工程里的位置

字幕层不是独立模块，它是像素通路上的**一个组合级**。下面的图是从 SD 卡到 HDMI 的完整链路，其中被本手册覆盖的部分用深色标出。

为什么字幕层必须在 `video_overlay` 内部

`video_overlay` 已经维护了 8 个图层的优先级链。字幕若做成独立模块、在 `video_overlay` 之后单独合成，就要**把整条优先级链再实现一遍**——因为它同样要判断「这一像素九宫格画不画、电平条画不画」。放进 `video_overlay` 内部，只需在链里插一级，优先级关系自然继承。

2.2 三层职责

这个切分的直接好处是：**验证可以分层做**。字形对不对，靠目视核对点阵图（图 3）；文案对不对，靠比对字符串表清单（表 9）；坐标对不对，靠逐点比对参考模型（12 节）。三者互不干扰——坐标算错了不会被误诊成「字形错了」。

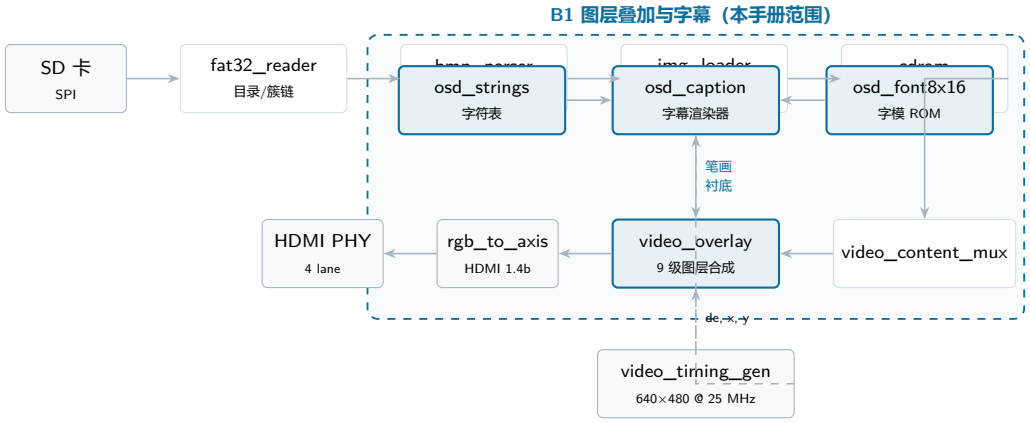


图 1: 像素通路。字幕层夹在 video_content_mux 与 video_rgb_to_axis 之间，在 video_overlay 内部完成合成，全程无缓冲。

表 1: 三个模块的分工。每一层只做一件事，任何一层换掉不影响其余两层。

模块	输入 → 输出	职责边界
osd_caption	坐标 → 笔画/衬底	**唯一的坐标逻辑所在。**判断当前像素属于哪块文字、算出行号列号位号。它不知道字符长什么样，也不知道文案是什么。
osd_strings	{行, 列} → ASCII	**文案表。**16 行 × 64 列的只读查表。改文案只需改这一层（且应当通过脚本改）。
osd_font8x16	{字符, 行} → 8 bit	**字形表。**41 个字符的字模。改字形只需改这一层（且应当通过脚本改）。

3 端口定义

3.1 osd_caption（字幕渲染器）

时钟域与复位约定

osd_caption **没有跨时钟域逻辑**，全部寄存器（scroll_px、scroll_div）都挂在 pixel_clk 上，与视频时序发生器同源。复位是**异步复位、同步释放**的常规写法（always @(posedge pixel_clk or negedge rst_n)）。若在别的设计里复用，注意 rst_n 必须与 pixel_clk 所属时钟域一致，不要接系统复位——否则滚动相位会在复位释放时与视频时序错位一帧（视觉上标语从中间开始）。

3.2 osd_strings 与 osd_font8x16

两者都是**纯组合查表**，无时钟、无复位。这一点很关键：它们可以被放在任意位置，不会引入延迟，也不需要复位序列。

3.3 在 video_overlay 里怎么接

video_overlay 为此新增了三个端口，插在 wave_en 之后、rgb_out 之前：

3.4 状态机编码到状态词的映射

这是集成时最容易接错的地方：playback_state 是 3 bit 状态编码，**不是行号**，必须经过映射。

表 2: osd_caption 端口表。方向以模块为参照：→ 输入，← 输出。

	信号	位宽	范围	说明
→	pixel_clk	1	--	像素时钟 25 MHz
→	rst_n	1	--	异步复位，低有效
→	de	1	--	数据有效
→	active_x	11	[10:0]	当前像素列 0..639
→	active_y	10	[9:0]	当前像素行 0..479
→	caption_en	1	--	字幕层总开关
→	state_line	4	[3:0]	状态行行号，取 2..7（越界则显示空白）
→	index_val	5	[4:0]	页面序号 0..31，供 # / * 替换
←	caption_pixel	1	--	本像素是字幕笔画
←	caption_band	1	--	本像素落在字幕带内（用于压暗衬底）

表 3: 两级查表的端口。注意 osd_font8x16 的索引是 char_code[5:0] 而非 char_code - 0x20，理由见5 节。

	信号	宽	说明
→	line_sel	4 [3:0]	文本行 0..15
→	col	6 [5:0]	字符列 0..63
←	char_code	7 [6:0]	该位置的 ASCII 码，0x00 = 无字

	信号	宽	说明
→	char_code	6 [5:0]	字符码低 6 位（ASCII & 0x3F）
→	row	4 [3:0]	字形行号 0..15（0 = 最上一行）
←	pixels	8 [7:0]	该行 8 个像素，bit7 = 最左

表 4: video_overlay 新增端口。三者都是**组合输入**，没有时序要求。

信号	位宽	说明
caption_en	1	字幕层总开关。顶层**常接 1'b1**（理由见 10 节）
play_state	3	播放状态机编码，用于选状态词行号（映射见表 5）
auto_play	1	自动轮播开关，用于区分 PAUSE 与 AUTO

表 5: 状态映射真值表。注意 ST_AUTO 与 ST_READY+auto_play 显示不同的词——两者对用户含义不同，不能合并。

编码	状态名	auto_play	行号	显示文字	含义
3'd0	ST_IDLE	–	5	WAITING	上电待机
3'd1	ST_SCAN	–	5	WAITING	扫描 SD 卡
3'd2	ST_LOAD	–	5	WAITING	正在解码图片
3'd3	ST_READY	**0**	3	PAUSE	就绪但未轮播
3'd3	ST_READY	**1**	4	AUTO	就绪且轮播已开
3'd4	ST_AUTO	–	2	PLAY	正在逐张自动播放
3'd5	ST_EMERG	–	6	EMERGENCY	应急模式
3'd6	ST_FAIL	–	7	FAIL	读图失败
default	–	–	5	WAITING	越界兜底

为什么 ST_AUTO 和 ST_READY+auto_play 不能都显示 AUTO

ST_AUTO 是「机器正在自己翻页」，此时画面每几秒变一次，用户什么都不用做。ST_READY 且 auto_play=1 是「机器停在那里等你按，但轮播开关已经打开」。前者是**正在进行时**，后者是**准备就绪**。如果都显示 AUTO，用户看到 AUTO 却不知道现在该不该等——一个显示「正在跑」，一个显示「随时会跑」，这是两种不同的用户动作指引。

4 屏幕坐标与版式

4.1 四块文字的位置

这是整个模块最需要精确的一张表。所有坐标以 640×480 有效区的左上角为原点，x 向右、y 向下。

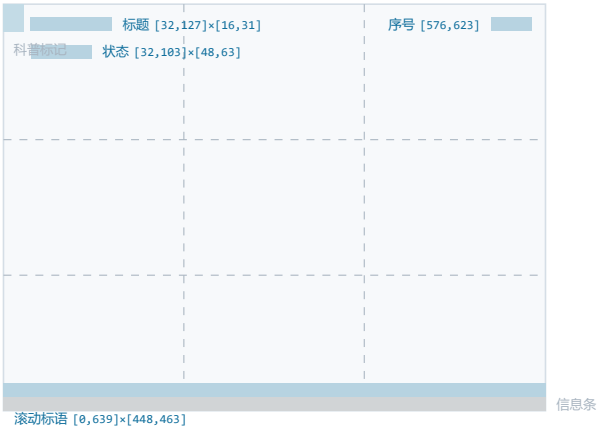


图 2: 版式。浅蓝块是四块字幕区域；虚线是既有九宫格参考线；深色是既有信息条。标题从 $x = 32$ 起是为了给左上角的科普标记（$x \le 23$）让位。

表 6: 字幕版式参数表。注意所有 y 带起点都是 16 的整数倍——这不是巧合，见下文。

槽位	y_0	y_1	x_0	x_1	行号 / 内容
标题	16	31	32	127	行 0：SMART CAMPUS
序号	16	31	576	623	行 1：IMG #*
状态	48	63	32	103	行 2..7：状态词
标语	448	463	0	639	行 15：64 字符滚动

4.2 三个坐标设计决策

决策一：y 带起点全是 16 的整数倍

字形有 16 行，行号 row 本应是 active_y - y_0。但若 y_0 是 16 的倍数，则 row = active_y[3:0] 直接成立——取低 4 位就等于取模 16，减法是免费的。

$16 = 16 \times 1$、$48 = 16 \times 3$、$448 = 16 \times 28$，三条带全部满足。省下 4 个 4 位减法器。这不是「省一点点」，而是**让代码里不出现减法**，从而不会有人在改坐标时忘记同步改减法。

决策二：标题从 $x = 32$ 起，给科普标记让位

科普模式（KEY3）在左上角画一块 $x \le 23$、$y \le 31$ 的青色标记，它是「当前处于科普态」的**唯一常显指示**。如果标题从 $x = 0$ 起，就会把它盖掉——KEY3 按下去屏幕毫无变化，用户会以为按键坏了。从 32 起，两者之间留 8

像素空隙。

决策三：标语放 $y = 448$，紧贴信息条上方

底部信息条（KEY12）占 $y \geq 464$。标语占 $[448, 463]$，两块不重叠，**可以同时打开。**若标语放在 $y = 464$ 以下，两者就互斥了——而「底部状态条 + 滚动标语」是正常的同屏组合。

4.3 列号换算：最容易出错的一处

从像素 x 算字符列，直觉写法是 `(x - X0) >> 3`。为了省减法器，会写成 `(x >> 3) - (X0 >> 3)`。这两者**不一定等价。**

位切片会折返——综合与仿真都不报错

Verilog 里 `active_x[8:3]` 取的是 `(x >> 3) mod 64`，不是 `x >> 3`。

- 标题/状态：$x \in [32, 127]$，$(x \gg 3) \in [4, 15]$，`x[6:3]` 取模 16 不折返，减 4 正确。
- 序号：$x \in [576, 623]$，$(x \gg 3) \in [72, 77]$。用 `x[8:3]`（模 64）会折成 $[8, 13]$，再减 72 得负数（6 位无符号回绕成 $[56, 63]$）——查表查到完全无关的行。**必须用 `x[9:3]`（模 128）再减 72。**

这个 bug 在综合里是合法的（就是一张不同的查找表），在仿真里也过（如果 testbench 恰好只测了折返区之外的点）。它只在屏幕上表现为「页码显示成乱码字母」。

表 7: 列号换算的位切片选择。判断依据：位切片取模的模数必须大于该区间的最大值。

槽位	x 区间	$(x \gg 3)$ 区间	应使用的切片	减数
标题	$[32, 127]$	$[4, 15]$	`x[6:3]`（模 16）	4
状态	$[32, 103]$	$[4, 12]$	`x[6:3]`（模 16）	4
序号	$[576, 623]$	$[72, 77]$	`x[9:3]`（模 128）	72

经验法则：位切片的高位下标 k 要满足 $2^{k-2} > \max(x) \gg 3$。对序号，$\max = 77$，需要 $2^{k-2} > 77$，即 $k - 2 \geq 7$，$k \geq 9$——所以是 `[9:3]`。

5 字形 ROM

5.1 字符集：只有 41 个

本 ROM **不收录全部 95 个可打印 ASCII**，只收录 41 个：

空格 - . / 0–9 : A–Z

理由有二：字幕只表达「大写字母 + 数字 + 少量标点」，不需要小写字母与符号；而当前设计 LUT 余量只剩三成多（综合后 68.89%），砍掉一半字符集是必要的取舍。

字符集可以扩，但要算清代价

每个字形是 $16 \times 8 = 128$ bit 常量。41 个字形 $= 5248$ bit。若扩到 95 个可打印 ASCII，是 12160 bit，约 2.3 倍。综合成 LUT 查表后，实际占用增长通常小于线性（多路选择器可共享），但仍需重跑综合确认 LUT 不越界。扩字符集的正确做法是改 `gen_osd_font8.py` 的 `CHARSET` 常量后重新 `--emit`，不要手写 `.v` 里的 `case` 项。

5.2 字符集总表

下面的点阵图**直接从 RTL 的 `case` 常量解析生成**（13 节），不是手画的示意。每个字形下方是它的 ROM 索引（`char_code[5:0]`）。

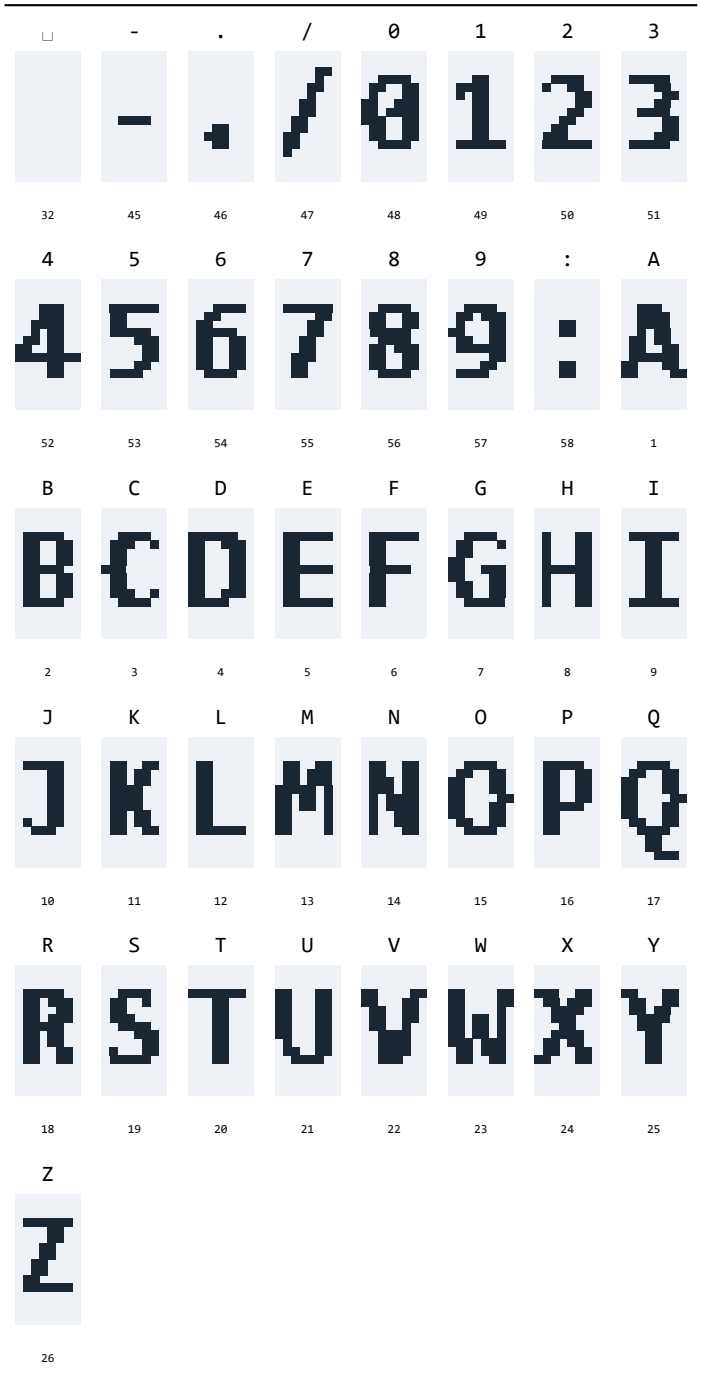


图 3: 41 个字形点阵。上方是字符，下方是 ROM 索引。注意 `B` 与 `8`、`D` 与 `0` 的区分度——这正是选 8×16 而非 8×8 的原因。

5.3 为什么索引是 `char_code[5:0]` 而不是 `char_code - 0x20`

收录的字符集全部落在 ASCII `0x20..0x5F` 区间内，且**低 6 位互不相同**：

表 8: 低 6 位唯一性。因为区间宽度恰好是 64，`ASCII & 0x3F` 是一一映射。

字符	ASCII	低 6 位	字符	低 6 位
空格	0x20	32	A	1
-	0x2D	45	Z	26
0	0x30	48	:	58
9	0x39	57	/	47

所以上层把字符码低 6 位直接接进来即可，**省掉一个减法器**，也省掉「减法结果越界怎么办」的边界判断。

未收录的索引走 `default: pixels = 8'h00`，屏幕上表现为**空白而不是乱码**——这是刻意的失败安全设计：占位符替换若失效，用户看到的是缺字，不是花屏。

5.4 为什么是 8×16 而不是 8×8

这是本项目实测得出的结论，不是凭经验选的。



图 4: 粒度对比。上排：用 8×8 栅格化时 A 的中宫与外围粘连成实心块、B/8、D/0 互相难分；下排：同一字体改用 8×16 后笔画拉开，五个字符都能分辨。

8×16 是经典 VGA 文本模式规格（720×400 文本模式下每字符 9×16），在 640×480 上做单行字幕，这个粒度正好：一行文字占 16 像素，既不显得细瘦，也不占掉太多画面。

5.5 为什么不用 BRAM

BRAM 有 1 拍读延迟。字幕必须与图像**同拍**组合出图——若字模走 BRAM，字幕会比图像晚 1 拍，屏幕上表现为文字整体右移一个像素且与背景错位。用 LUT 实现的组合查表没有延迟，代价是占用 LUT 而非 BRAM。

同理，若将来要用 16×16 汉字字模（数据量 16 倍），也不能简单换成 BRAM，需要重新设计成**预取一拍**的流水线，那时「同拍组合」这个前提就要放弃。

6 字符串表

6.1 16 行 × 64 列

地址 = `{line_sel[3:0], col[5:0]}`，共 10 bit。

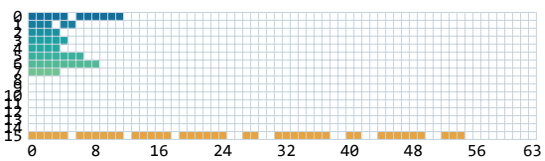


图 5: 地址映射。有色格 = 该位置有字符，灰色 = 空行（综合后完全消失）。行 15 满格是 64 字符的滚动标语；行 8–14 是预留空行。

为什么要留 8 行空的

行 8–14 全空，在综合后**完全不占资源**（case 里没有这些地址的项）。但留着它们，以后加新文案（比如加一行 SD ERROR、一行 LOADING）就不必重排行号——而重排行号意味着要同步改 osd_caption.v 里的 state_line 映射，那是跨文件改动，容易漏。空行的代价是零，收益是「以后加东西不用动别的文件」。

6.2 各行内容

表 9: 字符串表实际内容（从 RTL 解析生成）。· 表示该列是空格。最后一列是该行实际占用的列宽。

行	内容	字符数	列宽
0	SMART·CAMPUS	11	12
1	IMG·#*	5	6
2	PLAY	4	4
3	PAUSE	5	5
4	AUTO	4	4
5	WAITING	7	7
6	EMERGENCY	9	9
7	FAIL	4	4
15	SMART·CAMPUS·MED IA·CENTER····W ELCOME····ONLI NE··NOW	42	55

6.3 占位符 # 与 *

行 1 的文案是 IMG #*。其中 # 表示序号十位、* 表示个位，由 osd_caption 在查字模前替换。

为什么用 # 和 * 而不是别的

这两个字符**不在字模 ROM 的字符集内。**所以替换逻辑若失效（比如 state_line 接错、index_val 越界），ROM 查不到这两个码，输出空白——用户看到 IMG 后面缺两个字，一眼就知道是替换坏了。若用数字占位符，替换失效会显示成错误的数字，反而更难发现。

十位用三级比较得出（0..31 的十位只有 0/1/2/3 四种），个位用 index - (tens<<3) - (tens<<1) 即 index - 10*tens 求得，**避免引入除法器**——组合除法在 FPGA 上是几十个 LUT，为一个两位数页码不值得。

Listing 1: 十位与个位求值（osd_caption.v）

```
// 十位: 三级比较, 0..31 只有 0/1/2/3 四种
always @(*) begin
    if      (index_val >= 5'd30) idx_tens = 4'd3;
    else if (index_val >= 5'd20) idx_tens = 4'd2;
    else if (index_val >= 5'd10) idx_tens = 4'd1;
    else                         idx_tens = 4'd0;
end

// 10 x tens = (tens << 3) + (tens << 1), 避开除法器
wire [4:0] ten_tens = {1'b0, idx_tens, 3'b000} + {3'b000, idx_tens, 1'b0};
wire [4:0] idx_ones = index_val - ten_tens;

// 替换: '#'(0x23) -> 十位, '*'(0x2A) -> 个位
wire [6:0] rep_code = (raw_code == 7'h23) ? {3'b011, idx_tens} :
                      (raw_code == 7'h2A) ? {3'b011, idx_ones} :
                                            raw_code;
```

7 滚动标语

7.1 按像素滚，不按字符格滚

若按字符格滚（每 8 帧整格左移一格），看到的是**跳字**——文字一格一格地蹦。要平滑，必须按像素滚：整条标语左移 1 像素。

实现上就是把读表地址整体偏移：

Listing 2: 滚动坐标（osd_caption.v）

```
// 每 8 帧 +1 像素
wire frame_tick = de && (active_x == 11'd0) && (active_y == 10'd0);

// (x + scroll) >> 3 的低 6 位 = 字符列 mod 64
//   639 + 511 = 1150 < 2048, 11 位够, 不会溢出
wire [10:0] ticker_src = active_x + {2'b00, scroll_px};
wire [5:0]  ticker_col = ticker_src[8:3];

// 关键: 列内偏移必须用滚动后的坐标, 不是 active_x
wire [2:0]  t_bit = ticker_src[2:0];
```

7.2 三个必须踩对的点

一、列内偏移用 ticker_src[2:0]，不是 active_x[2:0]

列号偏移了 k 像素后，同一屏幕位置显示的是「原文本第 k 位开始的字符」，但**字形内部的第几位**也偏移了。若用 active_x[2:0]，文字会像被切成 8 列竖条再横向循环移动——每个字符内部的花纹原地不动，只有字符之间的顺序在变。视觉上是「碎掉的字」。

二、scroll_px 必须 9 bit（0..511），不能只 6 bit

若只用 6 bit，从 63 回绕到 0 时，整条标语会**瞬间跳 63 像素**——因为 $+1$ 在 6 bit 下变成了 -63。512 像素 = 64 字符 = 完整一轮，回绕时画面完全重合，看不出接缝。所以计数器宽度要覆盖一整轮，而不是「够存列号」。

三、帧基准用 de && (x==0) && (y==0)，不加 vs 端口

滚动计数器每帧递增一次，需要一个帧同步信号。用「有效区首像素」作为帧基准，不需要额外引入 vs 端口——每加一个顶层端口都要同步改 testbench 的 DUT 例化，而 de/active_x/active_y 已经在模块端口里了。

这个帧基准有一个前提

(active_x==0) && (active_y==0) 每帧只成立一次，这依赖 video_timing_gen 的行为：active_x/active_y 在消隐期是否回零。当前实现里它们在消隐期保持递增值，所以每帧只有一个像素同时满足 x=0 且 y=0。若将来改时序发生器让 x/y 在消隐期归零，这个条件会在消隐期持续成立，滚动会变成每帧跳很多像素。改用 vs 的下降沿更稳妥。

7.3 滚动速度

scroll_div 是 3 位分频，每 8 帧 $+1$ 像素：

$$\text{一轮时间} = 512 \text{ 像素} \times 8 \text{ 帧/像素} \div 59.5 \text{ 帧/秒} \approx 68.8 \text{ 秒}$$

约一分十秒走完一轮。这个速度是「能看清字、又明显在动」的折中：太快（比如 10 秒一轮）读不完一句，太慢（比如 5 分钟一轮）看起来像静止。

8 单路渲染架构

8.1 核心决策

屏幕上虽然有 4 处文字，但一个像素在任一时刻**只可能属于其中一处**——它们的 y 带互不重叠（标题/序号共用一条带，靠 x 区分）。所以只需要**一套** {字符串表 + 字模 ROM}，用 `if` 选出当前像素该查哪一行哪一列。

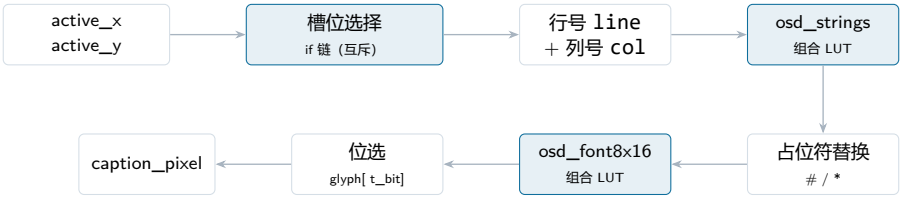


图 6: 单路渲染。四块文字共用一条查表通路，靠 `if` 链互斥选择。整条通路是**纯组合**的，从坐标到像素输出零延迟。

8.2 如果给每块文字各配一套 ROM 会怎样

四套 ROM 意味着四份 `case` 表同时存在。综合器**不会**把它们合并——因为四个表的地址来源不同、使能条件不同，逻辑上就是四个独立的多路选择器。粗略估算 LUT 占用会到原来的 3–4 倍，而当前 LUT 余量只有三成。单路渲染把这块代价压到最小。

代价是**这个模块的时序路径变长了**：坐标 → 槽位选择 → 查字符串表 → 替换 → 查字模 → 位选，全在一拍内完成。综合报告里字幕相关信号（`t_line_b`、`t_col_b`）确实出现在关键路径上，但 setup 余量仍有 $+1.935\,\text{ns}$，是安全的。

什么时候该放弃单路

若将来加更多文字槽位（比如每张图都有独立的说明文字），`if` 链会变长、路径会变慢。那时应该改成**两级流水**：第一拍算地址并读出字符码，第二拍查字模并输出像素——代价是字幕比图像晚 1 拍，需要在 `video_overlay` 里对图像做 1 拍延迟对齐。当前槽位数只有 4，不值得引入这个复杂度。

9 衬底：为什么压暗而不是填黑

9.1 问题

白色字幕（`0xF0F0F0`）在浅色画面（比如一张雪景、一页白纸照片）上**会消失**。必须有衬底。两种做法：

做法	效果	问题
填黑（`0x000000`）	字绝对清楚	画面被硬切出一条黑条，像坏点
压暗（RGB 各右移 2 位）	字清楚	保留画面明暗层次，像半透明底纹

选压暗。实现只有一行：

Listing 3: 衬底压暗（`video_overlay.v`）

```
// 压到 1/4 亮度：每通道右移 2 位（丢弃低 2 位，等价于除以 4）
wire [23:0] img_capped = {2'b00, r_inv[7:2], 2'b00, g_inv[7:2], 2'b00, b_inv[7:2]};
wire [23:0] img_final  = px_cap_band ? img_capped : img_layer;
```

右移 2 位在 FPGA 上**不消耗任何逻辑**——就是重新接线，不需要移位器。

9.2 衬底只覆盖文字实际占据的 x 区间

`caption_band` 不是整条 y 带通栏，而是只在四块文字的实际 x 范围内有效。标题带 $y \in [16, 31]$ 上，只有 $x \in [32, 127]$（标题）与 $x \in [576, 623]$（序号）有衬底，中间 $x \in [128, 575]$ 是原始画面。

这样做的原因：如果整条 y 带通栏压暗，屏幕上会出现两条贯穿全宽的暗带，即使那一段没有文字。视觉上像「屏幕上有污渍」。

9.3 优先级链

字幕在 9 级优先级里排第 7 位，只在信息条之上、图像之下：

表 10: 图层优先级（从高到低）。字幕排在信息条之上，是因为信息条是「状态条」，字幕是「信息」——两者同时出现时，文字应当可见。

级	图层	颜色	开关
1	九宫格参考线	`00FF00` 纯绿	KEY11
2	科普态标记	`00C8E0` 青	KEY3
3	信息条方块	`40E0D0` 青绿	KEY12
4	音频电平条	`20FF20` 亮绿	KEY13
5	音频波形	黄	KEY13
6	信息条底条	`101820` 深蓝黑	KEY12
7	**字幕笔画**	`F0F0F0` 近白	`caption_en`
8	**字幕衬底（压暗）**	图像 $\div 4$	`caption_en`
9	图像本身	–	常开

10 为什么字幕层常接 `1'b1`

顶层把 `caption_en` 直接接成 `1'b1`，没有做按键开关。三个理由：

1. **性质不同。**九宫格、电平条是**临时调试**叠加，用完要关掉看真实画面。标题、页码、状态、标语是**设备常驻信息**——真实产品里不会给「关掉标题」的开关。
2. **键位已满。**板上有 16 个键，全部已分配：KEY0–3 播放控制、KEY4/5 测试图案、KEY6 静音、KEY7–14 各类开关、KEY15 应急。再抢一个键意味着推翻既有键位分配（工程里已经发生过一次按键冲突事故）。
3. **有替代方案。**真要临时看无字幕画面，改顶层一行即可，不需要为它常驻一个键。

如果将来要加开关

把 `caption_en` 接到 `key_action` 的一个新输出即可，但注意：`key_action` 的 testbench 有断言「KEY11/12/13/14 各按一次后所有开关回 0」（`tb_key_action.v` 的 K06 用例）。新增开关必须同步加进那条断言，否则回归会判负。

11 参数速查

改坐标时的检查清单

1. y_0 是 16 的倍数吗？不是 → 要加 `active_y - Y0` 减法器。
2. x_0 是 8 的倍数吗？不是 → 列号换算会错位。
3. 位切片 `active_x[k:3]` 的模数 2^{k-2} 大于 $x_1 \gg 3$ 吗？不是 → 会折返。
4. 与既有图层（九宫格、科普标记、信息条）的坐标重叠吗？
5. 文案长度 $\leq$ 区域宽度 $\div 8$ 吗？超了会被裁掉。

12 验证

表 11: 全部可调参数。改这些值必须同步检查对应约束。

参数	当前值	位置	改它要注意
`TITLE_Y0/Y1`	16 / 31	`osd_caption.v`	起点必须是 16 的倍数（否则要加减法器）
`STATE_Y0/Y1`	48 / 63	`osd_caption.v`	同上
`TICKER_Y0/Y1`	448 / 463	`osd_caption.v`	同上；且不能与信息条 $y \geq 464$ 重叠
`TITLE_X0/X1`	32 / 127	`osd_caption.v`	起点必须是 8 的倍数；且 $x_0 > 24$ 以避让科普标记
`INDEX_X0/X1`	576 / 623	`osd_caption.v`	位切片模数要覆盖 $x_1 \gg 3$
`STATE_X0/X1`	32 / 103	`osd_caption.v`	最宽状态词 `EMERGENCY` 是 9 字符 = 72 像素
`LINE_TICKER`	15	`osd_caption.v`	与 `osd_strings` 的行号对应
`scroll_div` 位宽	3 bit	`osd_caption.v`	决定滚动速度：2^n 帧/像素
`scroll_px` 位宽	9 bit	`osd_caption.v`	**必须是 9**，见 7 节
`CHARSET`	41 字符	`gen_osd_font8.py`	改后重跑 `--emit`
`LINE_TICKER` 长度	64	`gen_osd_strings.py`	必须正好 64（列号取模）

12.1 三档验证口径

工程里统一用三档标注，**不得混用**：

表 12: 验证档位与本次实际达到的程度。

档位	状态	内容
仿真	**已通过**	ModelSim 逐点比对；前端回归 20 个 testbench 全绿
综合布线	**已通过**	安路 TD 综合布线，时序满足，位流已生成
上板	**未做**	**尚未在实际硬件上确认观感**

12.2 逐点比对方法

坐标逻辑的验证不用「跑几个例子看输出对不对」——那样覆盖率极低。用**参考模型逐点比对**：testbench 里写一个独立的参考模型，对 640×480 的每个像素算一遍「应该是什么」，与 DUT 输出比对。

关键设计：参考模型**用除法与取模**（`/`、`%`）实现坐标换算，而 RTL 用位切片。两种写法刻意不同——这样位切片一旦折返或错位，比对立刻失败。若参考模型也照抄位切片，就变成「用同一套错误验证自己」。

12.3 预览图交叉校验

仿真验证的是「逻辑对不对」，验证不了「看起来对不对」。所以另写了一个 Python 渲染器，用**同一套字模与字符表数据**画出 PNG，再做两级校验：

- **静态三条带用 OCR**：把 PNG 按 8×16 网格切回字形，在 RTL 字模表里找唯一匹配。匹配不上 $\rightarrow$ 渲染管线与 RTL 数据不一致。
- **滚动用位移不变性**（不用 OCR）：`scroll=N` 的笔画掩码必须等于 `scroll=0` 掩码平移 N 像素。这条判据更强——它不关心「字是什么」，只要求「滚 N 像素 = 真的平移了 N 像素」。

表 13: 验证用例与结果。

ID	用例	内容	结果
C01	字模内容抽查	抽查若干 {字符, 行} 的点阵值	通过
C02	字符串表抽查	抽查若干 {行, 列} 的字符码	通过
C03	标题带逐点	$x \in [32, 127]$，$y \in [16, 31]$ 全像素	通过
C04	序号带逐点	4 个不同序号值	通过
C05	状态带逐点	6 种状态词各一遍	通过
C06	标语带	`scroll`=0 与 `scroll`=3	通过
C07	带外透明	字幕区外的像素不受影响	通过
C08	总开关关闭	`caption_en`=0 时全静默	通过
C09	滚动计数器	8 帧 +1、511 回绕、复位清零	通过
C10	衬底带几何	衬底只在文字实际 x 区间	通过
合计：56 条断言，3057 个像素逐点比对，零失配			**通过**

为什么滚动不能用 OCR

滚动偏移非 8 的整数倍时，屏幕上同一格会横跨两个字符，按格切分读出的是字模表里不存在的图案。这是**校验方法的局限，不是 RTL 的问题**。改用位移不变性后，9 个相位（1/3/7/8/17/64/180/255/511）共 10176 像素零失配。

12.4 综合布线结果

表 14: 资源占用（安路 TD，`flow_noise_sci_fix.tcl`）。字幕子系统共占用约 958 个 LUT。

资源	用量	总量	占比
LUT	13 502	19 600	68.89%
寄存器	5 993	19 600	30.58%
BRAM9K	17	64	26.6%
DSP	15	29	51.72%
时序：setup +1.935 ns，hold +0.003 ns			（均为正，满足）

网表落地确认：时序报告里有 42 处命中 `caption` 的路径，包括 `osd_caption.v`、`osd_strings.v`、`osd_font8x16.v` 三个源文件的行号引用——证明三层都真的进了网表，没有被综合器优化掉。

13 生成脚本

13.1 为什么字模和字符串表都必须脚本生成

手写这两种表有两个无法避免的错误源：

1. **数错列。**字符串表里 `SMART CAMPUS` 的 `C` 在第 6 列，手写 `10'd6` 时数错一格，**综合不报错、仿真不报错**，只有上板才看到「文字里多了一个空格」。这类错误在评审时也看不出来。
2. **字形抄错。**8×16 点阵的十六进制值手抄必错，而且错了之后「看起来像另一个字」，很难定位。

脚本生成的直接好处：**改文案只需改一个字符串常量**，列号自动展开。

13.2 gen_osd_font8.py

从 C:\Windows\Fonts\consolab.ttf（Consolas Bold）栅格化。关键参数：

参数	值	作用
FONT_SIZE	14	字号
BASE_Y	12	基线位置（anchor="ms"）
CELL_W / CELL_H	8 / 16	单元格尺寸
THRESHOLD	100	二值化阈值

两个开关：--art 打印 ASCII art 供目视核对，--emit 写入.v。**改字形必须先用 --art 看一眼**——8×16 下相邻字符的区分度只能靠眼睛判断，脚本算不出来。

13.3 gen_osd_strings.py

把字符串常量按 {line，col} 展开成 case 项。内置校验：字符集白名单、静态行长度 ≤ 16、标语正好 64 字符。超限直接报错退出，不会生成越界的表。

13.4 gen_datasheet_assets.py（本手册的图源）

本手册里的点阵图、地址映射图、端口表**全部由这个脚本从 RTL 解析生成**，不是手画的。这样做的理由：手册与代码不会脱钩。改了字模或文案，重跑脚本，手册里的图自动更新——如果手册里的图和代码不一致，读者按手册去改代码必然出错。

Listing 4: 重新生成手册资产

```
cd D:\FPGA_Compelate\SmartCampusMedia
python scripts/gen_datasheet_assets.py
```

14 踩坑清单

下面每一条都是本项目**实际踩过并修好**的，不是假设。

表 15: 已修复的缺陷。前三条在 RTL 里，后三条在验证脚本里。

#	现象	根因与修法	位置
1	页码显示成乱码字母	active_x[8:3] 在 $x = 576$ 折返。改用 [9:3]	RTL
2	文字像被切成竖条	列内偏移用了 active_x[2:0]。改用 ticker_src[2:0]	RTL
3	标语回绕时跳 63 像素	scroll_px 只 6 bit。改 9 bit	RTL
4	滚动出现半个字	预览脚本按「一格 8 像素共用一个列号」渲染。改逐像素	脚本
5	标语右侧 128 像素空白	预览脚本只画 64 格，RTL 是全宽 80 格	脚本
6	预览图里字母全成?	脚本把 ROM 索引当 ASCII 用。加索引 →ASCII 转换	脚本

三条最容易复发的

- **第 1 条**：任何「用位切片代替除法」的地方都要检查模数是否覆盖区间。综合与仿真都不会报这个错。
- **第 2 条**：任何「坐标偏移」都要问一句「偏移后的内部相位变了吗」。只改列号不改位号，是最隐蔽的错法。
- **第 3 条**：计数器位宽要覆盖**整个周期**，不是「够存当前值」。

15　还没做的

如实列出，避免读者误以为已完成：

1. **未上板。**综合布线通过、位流已生成，但没有在真实硬件上看过实际观感。仿真能证明逻辑对，证明不了「在显示器上好不好看」。
2. **不支持汉字。**当前只有 41 个 ASCII 字符。工程里另有 `science_font16.v`（16 × 16 汉字 ROM，40 字），但那是科普页专用的，字幕层没有接。要做汉字字幕，需要把字模换成 16 × 16、把字符码扩到 2 字节，并重新设计列号换算（每字符占 2 个 8 像素格）。
3. **字幕层无开关。**常接 `1'b1`，理由见 10 节。

如果要把这三条做完，顺序建议

先上板（成本最低，只需烧一次位流，能暴露所有观感问题）；再做汉字（工作量大，且要重新评估 LUT 余量）；开关放最后（纯锦上添花，且要改 testbench 断言）。

A　文件清单

表 16: B1 交付涉及的全部文件。

文件	行数	作用
`rtl/osd/osd_caption.v`	260	字幕渲染器（坐标逻辑）
`rtl/osd/osd_strings.v`	139	字符表（文案）
`rtl/osd/osd_font8x16.v`	750	字模 ROM（41 字形）
`rtl/video/video_overlay.v`	–	新增 3 端口、9 级优先级链、衬底压暗
`rtl/top/..._hdmi_top.v`	–	顶层接线
`sim/testbench/tb_osd_caption.v`	549	逐点比对（56 断言 + 3057 点）
`sim/testbench/tb_video_overlay.v`	659	集成验证（90 → 106 断言）
`scripts/gen_osd_font8.py`	–	字模生成
`scripts/gen_osd_strings.py`	–	字符表生成
`scripts/gen_datasheet_assets.py`	–	**本手册的图源**
`scripts/render_caption_preview.py`	–	预览图渲染
`scripts/verify_caption_preview.py`	–	预览图校验

B　字形来源与许可

字模由 `gen_osd_font8.py` 从 **Consolas Bold** 栅格化生成后固化为本 ROM。运行时**不加载任何字体文件**——ROM 里是已经烧死的位图。Consolas 随 Windows 分发，本项目仅将其字形栅格化为点阵使用。